

API Documentation and Postman Testing Report

Overview

The purpose of this task was to verify the functionality of backend APIs, perform API testing using Postman, validate request and response handling, and prepare API documentation for future development and integration.

The APIs tested in this phase include:

- User Profile APIs
- Product APIs
- User Measurements APIs
- Virtual Try-On APIs

The testing process ensured that all endpoints function correctly, return expected responses, handle invalid requests appropriately, and maintain proper data flow between the frontend, backend, and database.

API Verification Process

The following verification steps were performed for each API:

1. Endpoint Verification

- Verified that API routes are correctly configured.
- Checked route accessibility through Postman.
- Confirmed proper HTTP methods are used.

2. Request Validation

- Tested APIs using valid request data.
- Tested APIs using missing or invalid fields.
- Verified validation and error handling.

3. Database Verification

- Confirmed data insertion into MongoDB collections.
- Verified update and retrieval operations.
- Checked consistency between request data and stored records.

4. Response Verification

- Confirmed successful response status codes.
- Verified response body structure.
- Ensured meaningful error messages are returned.

5. Authentication Verification

- Tested protected routes using JWT tokens.
- Verified unauthorized access restrictions.
- Confirmed token validation functionality.

Postman Testing Procedure

The following steps were used while testing APIs in Postman:

Step 1: Configure Request

- Select HTTP Method (GET, POST, PUT, DELETE)
- Enter API Endpoint URL
- Add Authorization Token (if required)

Step 2: Add Request Data

For POST and PUT requests:

- Open Body tab
- Select Raw
- Choose JSON format
- Enter request payload

Step 3: Send Request

- Click Send
- Observe API response

Step 4: Validate Results

Verify:

- Status code
- Response body
- Database changes
- Error handling

Required Postman Screenshots

The following screenshots were captured for documentation purposes:

Successful Requests

- GET request success
- POST request success
- PUT request success
- DELETE request success

Error Handling

- Missing required fields
- Invalid input data
- Unauthorized access
- Resource not found

Authentication

- Valid JWT token access
- Invalid token access
- No token access

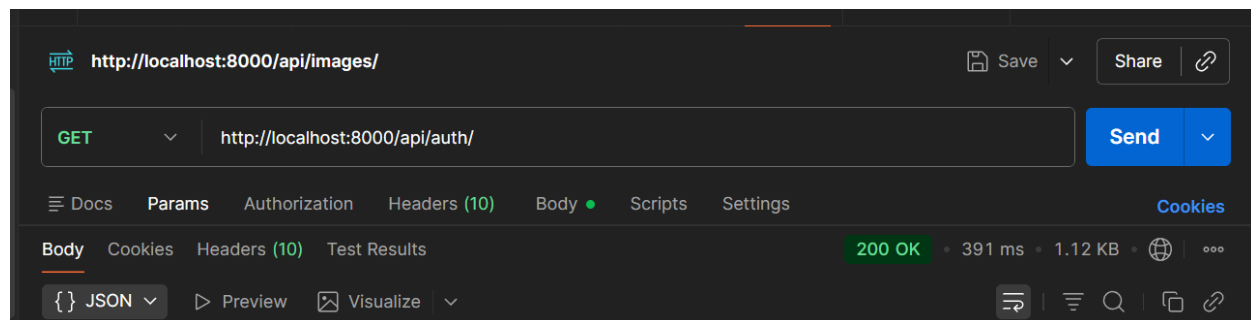
User Profile API Documentation

Purpose

The User Profile APIs manage user profile information, preferences, avatar data, and profile updates.

Get User Profile

Endpoint



Donapati Vishnu Vardhan Reddy
Bharath kumar

Authentication

Not Required

Success Response

```
65     {
66         "profileImage": "",
67         "avatarImage": "",
68         "bodyScanImage": "",
69         "_id": "6a0ebaecf4dba41b08178721",
70         "name": "John Doe",
71         "email": "john@example.com",
72         "password": "$2b$10$c2EBLSBv6bG83Cr.nAdKTOGbT9j4h6GCTJ/63EPwOHMLvsdisCnBC",
73         "__v": 0
74     },
75     {
76         "_id": "6a154c4b57b9020f4efcf0b6",
77         "name": "testuser",
78         "email": "test@example.com",
79         "password": "$2b$10$THcex/aDvj.L.CzfcBBIX./wmGcBYMf0y5HCS1sgcCu6v1t3AzVBC",
80         "profileImage": "",
81         "avatarImage": "",
82         "bodyScanImage": "",
83         "createdAt": "2026-05-26T07:31:23.931Z",
84         "updatedAt": "2026-05-26T07:31:23.931Z",
85         "__v": 0
```

```
82         "bodyScanImage": "",
83         "createdAt": "2026-05-26T07:31:23.931Z",
84         "updatedAt": "2026-05-26T07:31:23.931Z",
85         "__v": 0
86     },
87     {
88         "_id": "6a15527a803283b5231f8d30",
89         "name": "jone",
90         "email": "jone@gmail.com",
91         "password": "$2b$10$K.ZFUWnpTYuCwaRnfBBWh.kCHHoCTr8nX907.lo0xoskoQjnS0060",
92         "avatarImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1779798111/raritone/
          avatar/bqgujosul5vdpkdjm7oq.png",
93         "bodyScanImage": "",
94         "createdAt": "2026-05-26T07:57:46.621Z",
95         "updatedAt": "2026-05-26T12:21:52.236Z",
96         "__v": 0,
97         "profileImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1779798084/raritone/
          profile/xljtupnlhhh1fiwtcv2x.png",
98         "profileImagePublicId": "raritone/profile/vqvhqjedpbojvyq7k9nj"
99     }
100 ]
101 }
```

Expected Result

Returns complete user profile information.

Donapati Vishnu Vardhan Reddy
Bharath kumar

Post User Profile

Endpoint

POST

http://localhost:8000/api/auth/register

Send

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Schema

Beautify

Authentication

Required

Request Body

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Schema

Beautify

```
1 {
2   "name": "Alex Carter",
3   "email": "alex.carter92@example.com",
4   "password": "N7!mQ4#xP9@kL2"
5 }
```

Success Response

Body

Cookies

Headers (9)

Test Results

201 Created

676 ms

343 B

🌐

⋮

{ } JSON

Preview

Visualize

⌵

↺

⌵

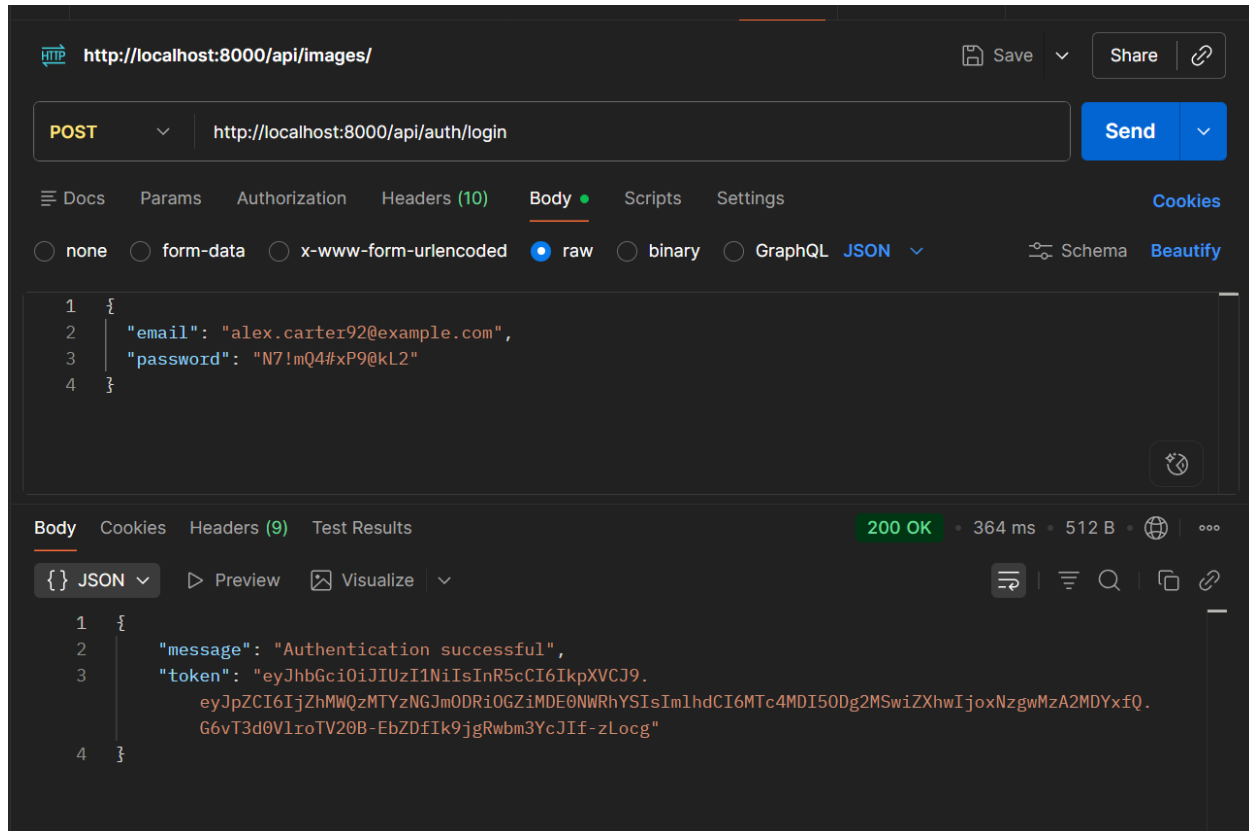
🔍

📄

🔗

```
1 {
2   "message": "User identity created successfully"
3 }
```

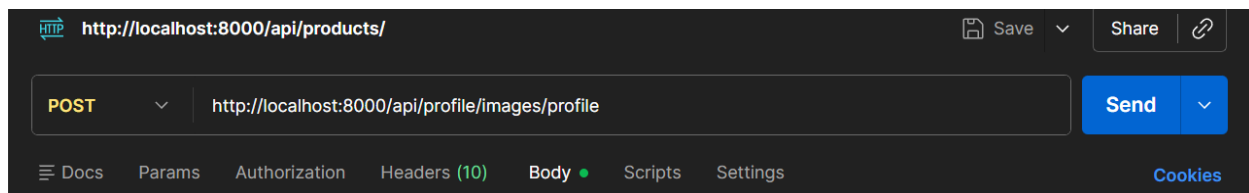
Post User Profile



Post Profile image

Endpoint

Post / api/profile/images/profile



Donapati Vishnu Vardhan Reddy
Bharath kumar

Authentication

Required

Request Body

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

☐ none

☒ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

	Key		Value	Description	...	Bulk Edit
☒	image	File	Screenshot (1).png			
	Key	Text	Value	Description		

Success Response

Body

Cookies

Headers (9)

Test Results

200 OK • 3.45 s • 991 B •

{}

JSON

Preview

Visualize

1

{

2

"success": true,

3

"message": "Profile image uploaded successfully",

4

"imageUrl": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780305110/raritone/profile/of42v19dbtqgo4dishct.png",

5

"user": {

6

"_id": "6a1d31634bf84b8fb0145daa",

7

"name": "Alex Carter",

8

"email": "alex.carter92@example.com",

9

"password": "\$2b\$10\$A/j6s8U2XJr5YRtnSaoIF.zsRfVeL7SHsW9Mks1sjTLQatQFuQ5D6",

10

"profileImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780305110/raritone/profile/of42v19dbtqgo4dishct.png",

11

"avatarImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780304708/raritone/avatar/zsqnlvhe6eudo71pwkx0.png",

12

"bodyScanImage": "",

13

"createdAt": "2026-06-01T07:14:43.995Z",

14

"updatedAt": "2026-06-01T09:11:51.905Z",

15

"__v": 0

16

}

17

}

Post Profile avatar

Endpoint

Post /api/profile/images/avatar

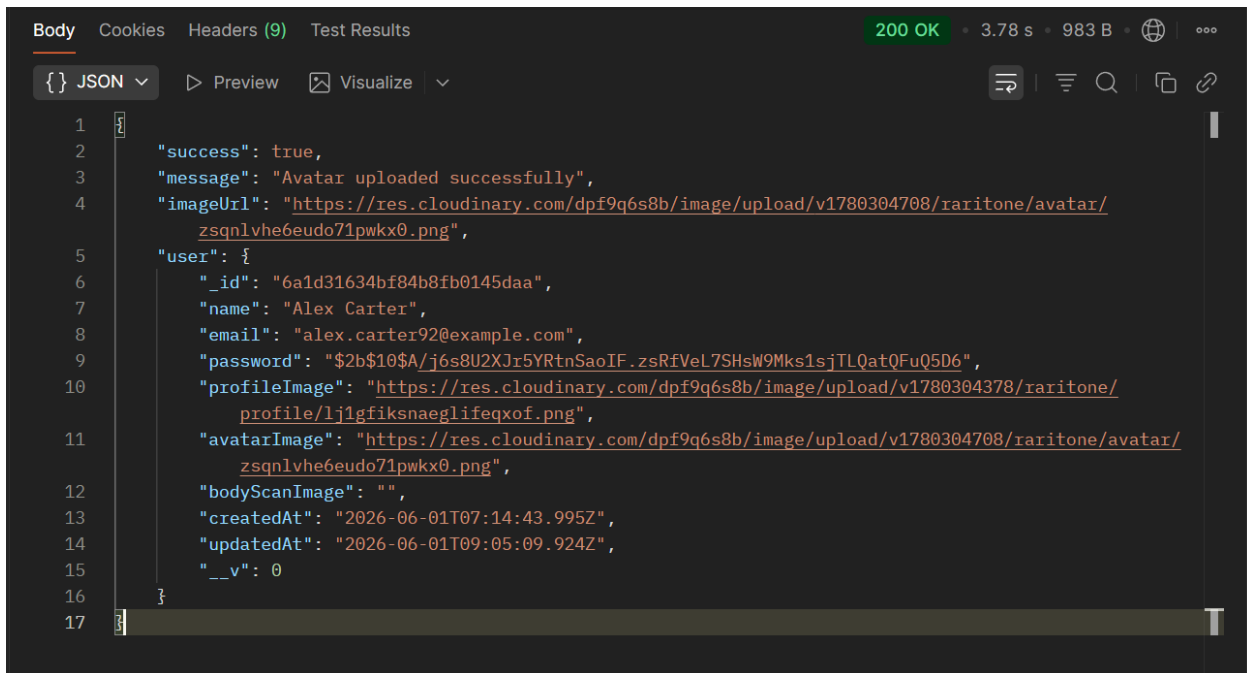
Authentication

	Docs	Params	Authorization	Headers (10)	Body	Scripts	Settings	Cookies
Headers 9 hidden								
	Key	Value	Description	...	Bulk Edit	Presets		
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....						
	Key	Value	Description					

Request Body

	Docs	Params	Authorization	Headers (10)	Body	Scripts	Settings	Cookies
Headers 9 hidden								
	Key	Value	Description	...	Bulk Edit	Presets		
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....						
	Key	Value	Description					

Success Response

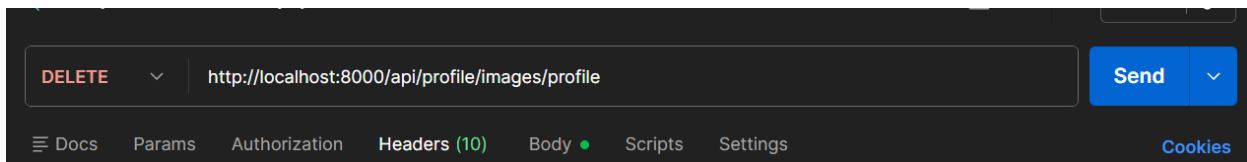


The screenshot shows a REST client interface with the 'Body' tab selected. The response is a JSON object indicating a successful upload. The status is 200 OK, with a response time of 3.78 s and a body size of 983 B. The JSON content is as follows:

```
1 {
2   "success": true,
3   "message": "Avatar uploaded successfully",
4   "imageUrl": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780304708/raritone/avatar/zsqnlvhe6eudo71pwkx0.png",
5   "user": {
6     "_id": "6a1d31634bf84b8fb0145daa",
7     "name": "Alex Carter",
8     "email": "alex.carter92@example.com",
9     "password": "$2b$10$A/j6s8U2XJr5YRtnSaoIF.zsRfVeL7SHsW9Mks1sjTLQatQFuQ5D6",
10    "profileImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780304378/raritone/profile/lj1gfiksnaeglifeqxof.png",
11    "avatarImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780304708/raritone/avatar/zsqnlvhe6eudo71pwkx0.png",
12    "bodyScanImage": "",
13    "createdAt": "2026-06-01T07:14:43.995Z",
14    "updatedAt": "2026-06-01T09:05:09.924Z",
15    "__v": 0
16  }
17 }
```

Delete profile

Endpoint



Authentication

Required

Request Body

Required

DocsParamsAuthorizationHeaders (10)BodyScriptsSettingsCookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key		Value	Description	...	Bulk Edit
☒	image	File	Screenshot (1).png			
	Key	Text	Value	Description		

Success Response

BodyCookiesHeaders (9)Test Results200 OK • 107 ms • 353 B • 000

{ } JSON

PreviewVisualize

```
1 {
2   "success": true,
3   "message": "Profile image deleted successfully"
4 }
```

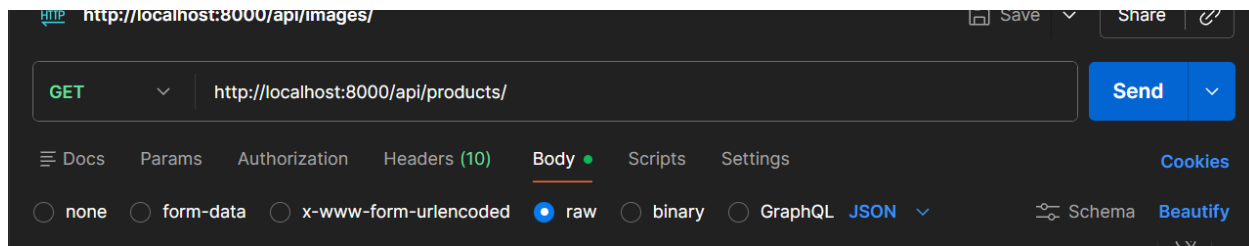
Product API Documentation

Purpose

Product APIs manage product listing, retrieval, creation, updating, and deletion

Get All Products

Endpoint



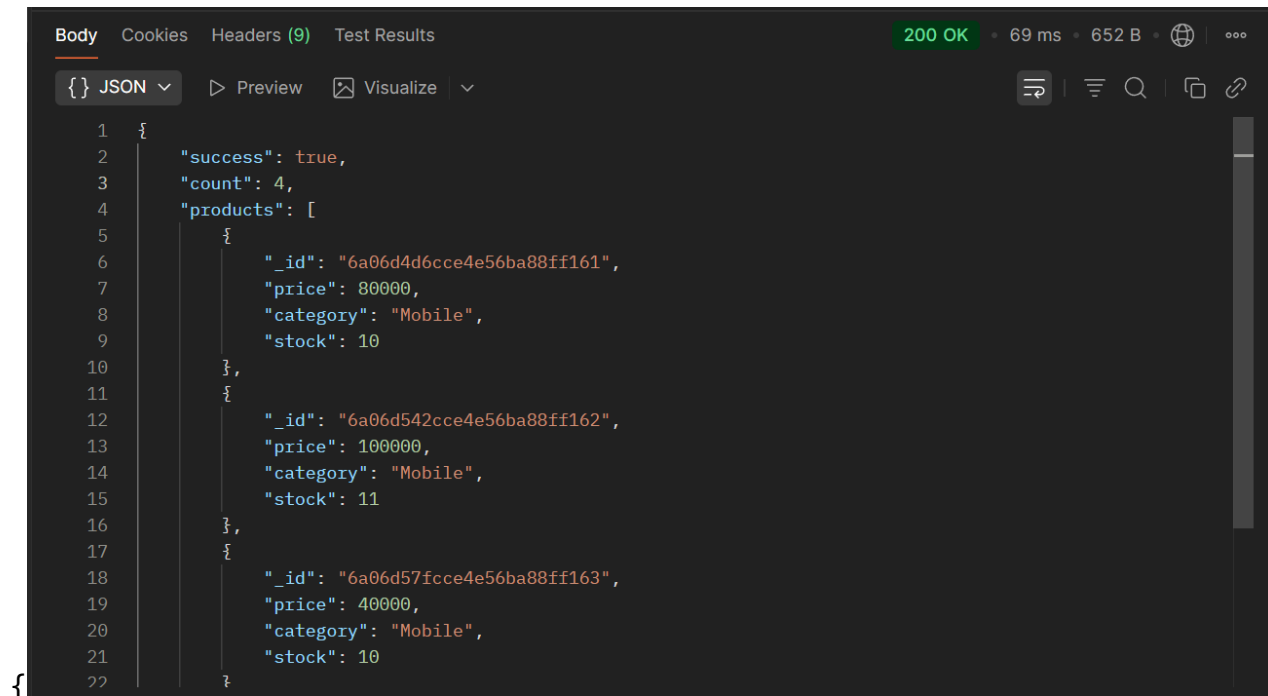
Authentication

Not Required

Request Body

Not Required

Success Response



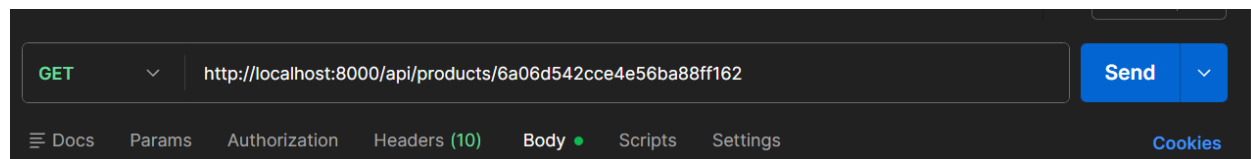
The screenshot shows a REST client interface with a dark theme. The top bar indicates a successful response with status 200 OK, a response time of 69 ms, and a body size of 652 B. The response is displayed in JSON format. The JSON structure is as follows:

```
1 {
2   "success": true,
3   "count": 4,
4   "products": [
5     {
6       "_id": "6a06d4d6cce4e56ba88ff161",
7       "price": 80000,
8       "category": "Mobile",
9       "stock": 10
10    },
11    {
12      "_id": "6a06d542cce4e56ba88ff162",
13      "price": 100000,
14      "category": "Mobile",
15      "stock": 11
16    },
17    {
18      "_id": "6a06d57fcce4e56ba88ff163",
19      "price": 40000,
20      "category": "Mobile",
21      "stock": 10
22    }
23  ]
24 }
```

Get Single Product

Endpoint

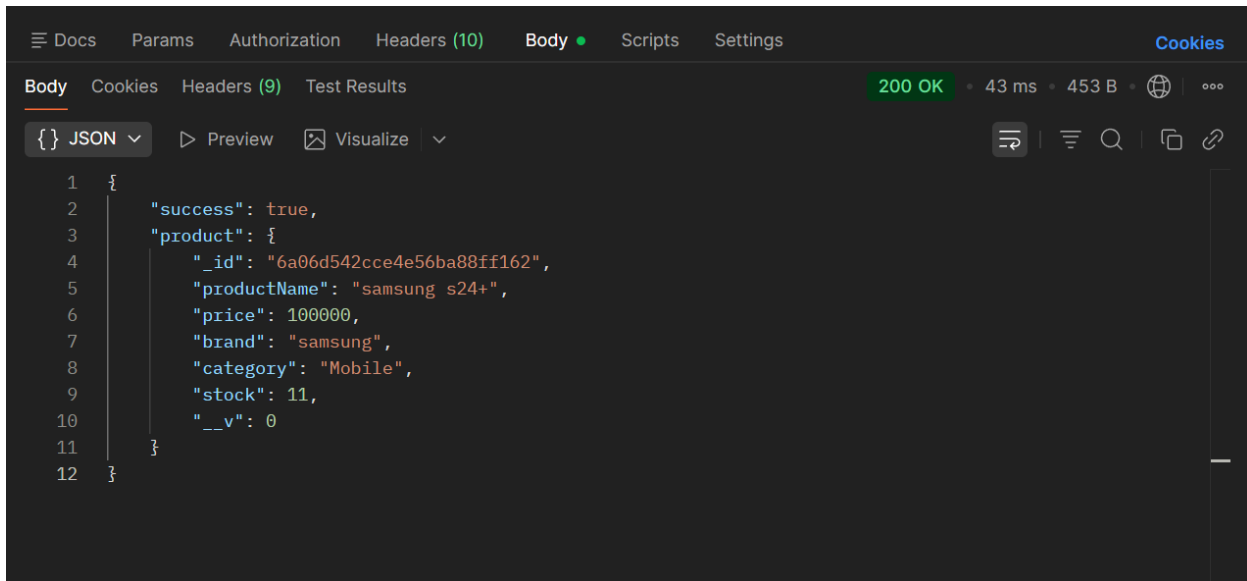
GET /api/products/:id



Authentication and Request body

Not Required

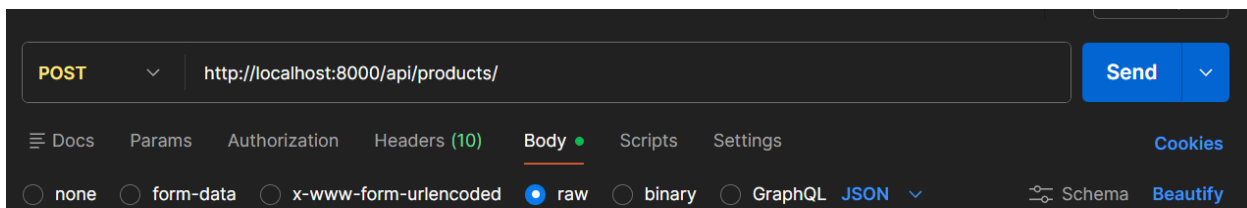
Success Response



Add Product

Endpoint

POST /api/products/



Authentication

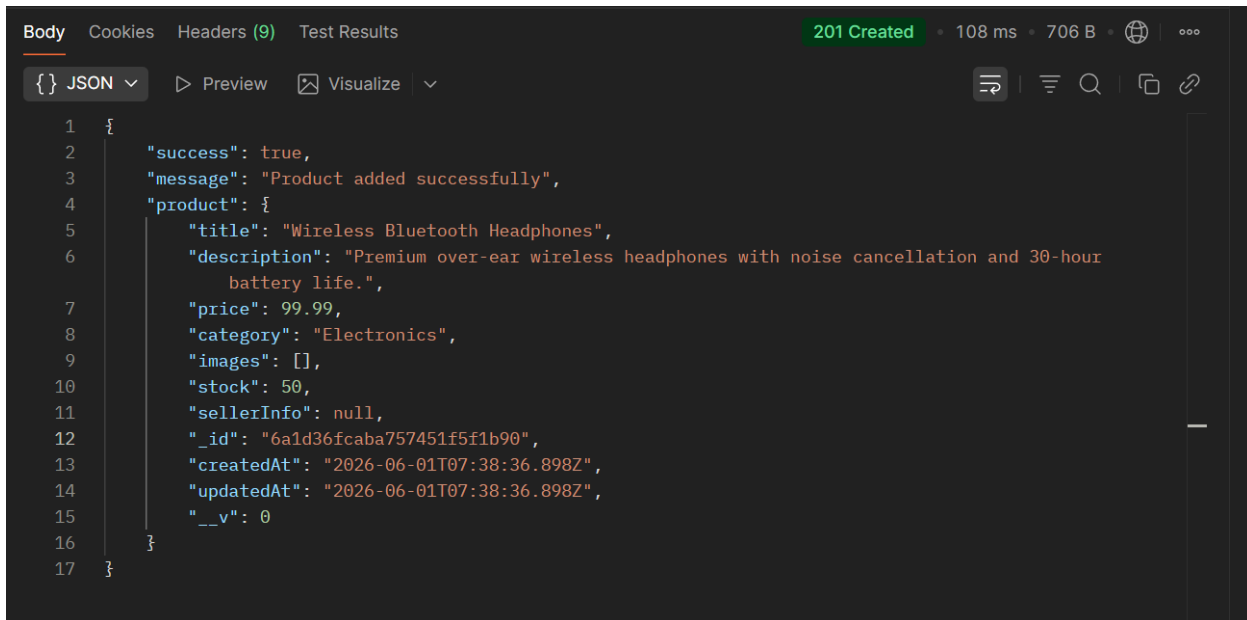
Required

Docs Params Authorization Headers (10) Body Scripts Settings Cookies			
Headers 9 hidden			
	Key	Value	Description
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....	
	Key	Value	Description

Request Body

Docs Params Authorization Headers (10) Body Scripts Settings Cookies			
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify			
1	{		
2	"title": "Wireless Bluetooth Headphones",		
3	"description": "Premium over-ear wireless headphones with noise cancellation and 30-hour battery life.		
4	,"		
5	"price": 99.99,		
6	"category": "Electronics",		
7	"images": [		
8	"https://example.com/images/headphones-front.jpg",		
9	"https://example.com/images/headphones-side.jpg"		
10	],		
11	"stock": 50,		
12	"sellerInfo": "TechStore Pvt Ltd"		
	}		

Success Response



The screenshot shows a REST client interface with the following details:

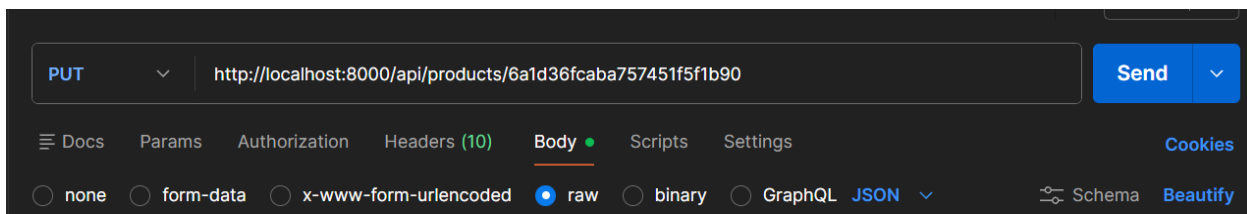
- Body** tab selected.
- JSON** format selected.
- 201 Created** status code.
- 108 ms** response time.
- 706 B** response size.
- Test Results** tab is also visible.

```
1 {
2   "success": true,
3   "message": "Product added successfully",
4   "product": {
5     "title": "Wireless Bluetooth Headphones",
6     "description": "Premium over-ear wireless headphones with noise cancellation and 30-hour battery life.",
7     "price": 99.99,
8     "category": "Electronics",
9     "images": [],
10    "stock": 50,
11    "sellerInfo": null,
12    "_id": "6a1d36fcaba757451f5f1b90",
13    "createdAt": "2026-06-01T07:38:36.898Z",
14    "updatedAt": "2026-06-01T07:38:36.898Z",
15    "__v": 0
16  }
17 }
```

Update Product

Endpoint

Put /api/products/:id



The screenshot shows a REST client interface with the following details:

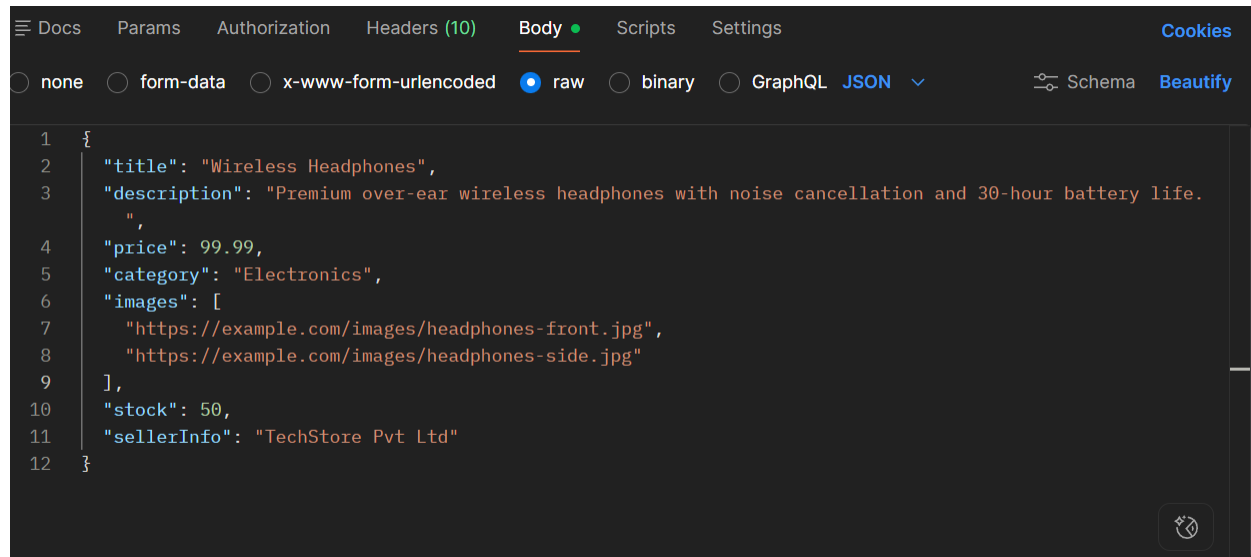
- PUT** method selected.
- http://localhost:8000/api/products/6a1d36fcaba757451f5f1b90** endpoint.
- Send** button.
- Body** tab selected.
- raw** format selected.
- Schema** and **Beautify** options.

Authentication

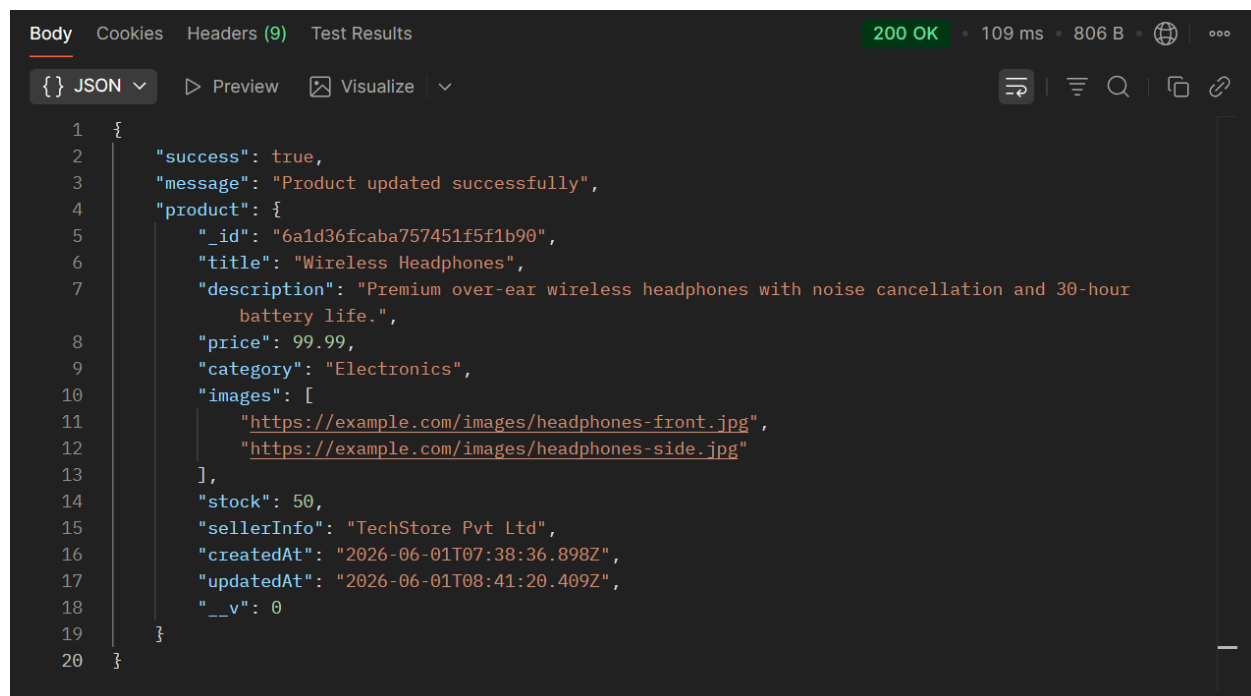
Required

Request Body

Required



Success Response



Purpose

Add Measurements

A screenshot of a REST client interface. At the top, there's a dark bar with a dropdown menu set to 'POST' and a text input field containing the URL 'http://localhost:8000/api/measurements/'. To the right of the URL bar are two blue buttons: 'Send' and a dropdown arrow. Below the URL bar is a horizontal menu with tabs: 'Docs', 'Params', 'Authorization', 'Headers (10)', 'Body' (which is selected and has a green dot), 'Scripts', and 'Settings'. To the far right of this menu is a blue button labeled 'Cookies'. Below the tabs is another row of options: radio buttons for 'none', 'form-data', 'x-www-form-urlencoded', and 'raw' (which is selected with a blue dot), followed by radio buttons for 'binary' and 'GraphQL', and a dropdown menu set to 'JSON'. To the right of these options are two more buttons: 'Schema' and 'Beautify'.

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

Headers

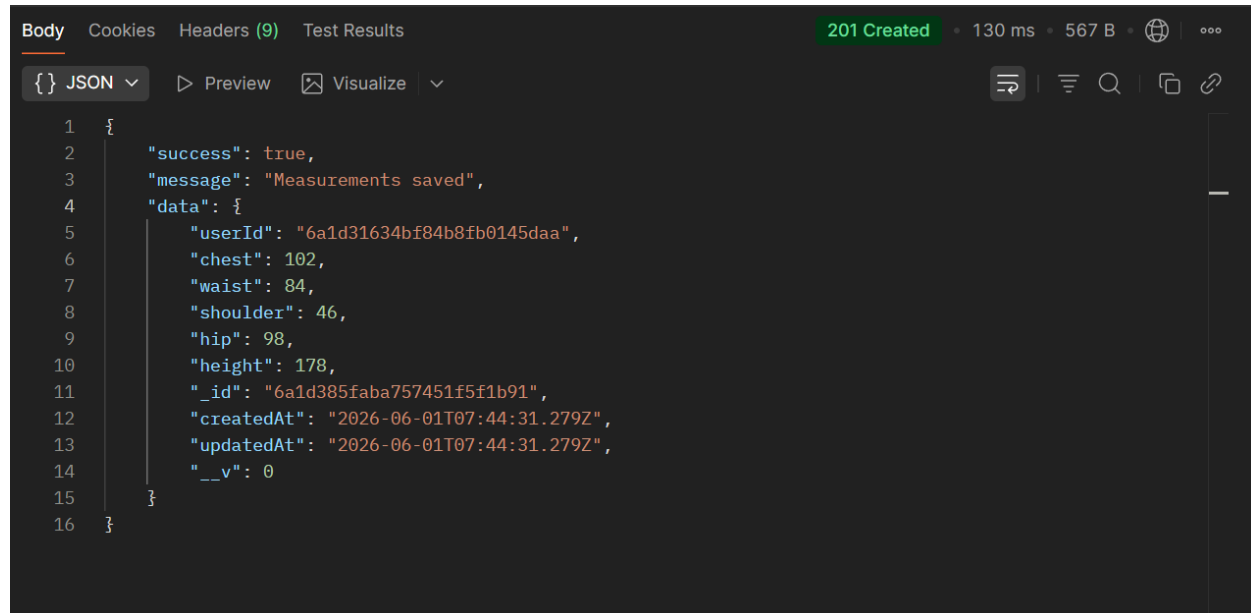
9 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☒	Authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....				
	Key	Value	Description			

Request Body

```
1  {
2    "userId": "665f1a2b3c4d5e6f7890abcd",
3    "chest": 102,
4    "waist": 84,
5    "shoulder": 46,
6    "hip": 98,
7    "height": 178
8  }
```

Success Response



The screenshot shows a REST client interface with the following details:

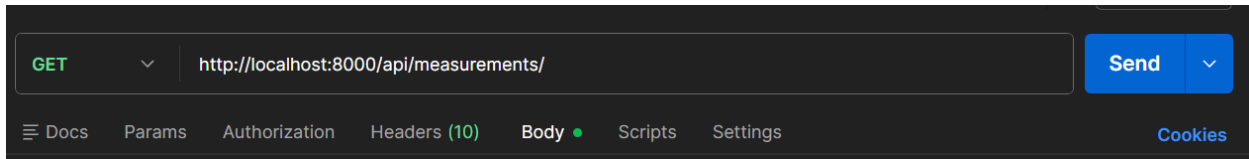
- Body** tab selected, showing JSON format.
- Status: **201 Created**, 130 ms, 567 B.
- Response body (JSON):

```
1  {
2    "success": true,
3    "message": "Measurements saved",
4    "data": {
5      "userId": "6a1d31634bf84b8fb0145daa",
6      "chest": 102,
7      "waist": 84,
8      "shoulder": 46,
9      "hip": 98,
10     "height": 178,
11     "_id": "6a1d385faba757451f5f1b91",
12     "createdAt": "2026-06-01T07:44:31.279Z",
13     "updatedAt": "2026-06-01T07:44:31.279Z",
14     "__v": 0
15   }
16 }
```

Get Measurements

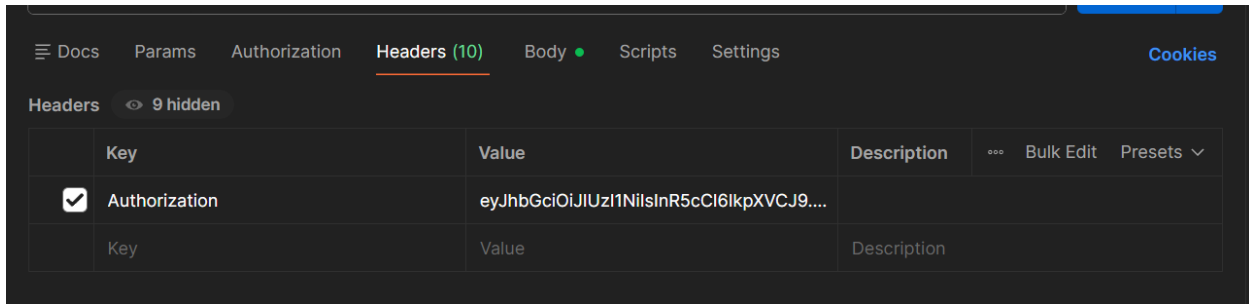
Endpoint

GET /api/measurements/



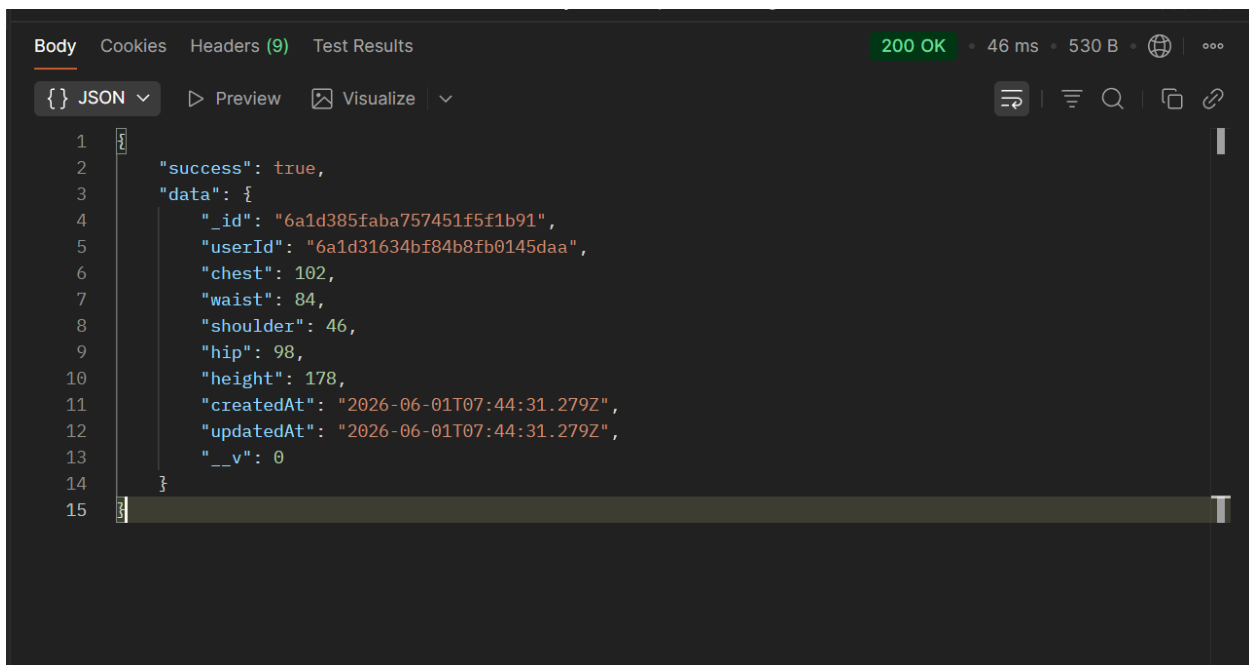
Authentication

Required



Request Body : Not Required

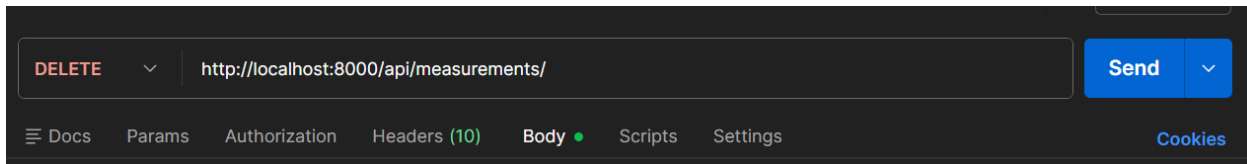
Success Response



Delete Measurements

Endpoint

delete/api/measurements/



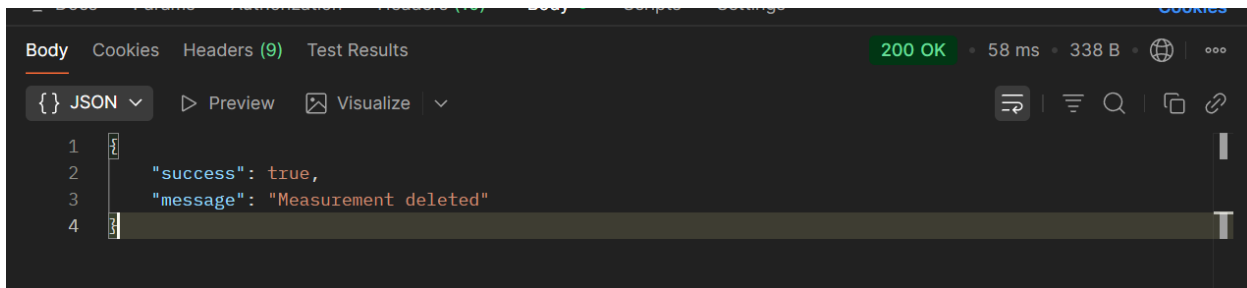
Authentication

Not Required

Request Body

Not Required

Success Response



Virtual Try-On API Documentation

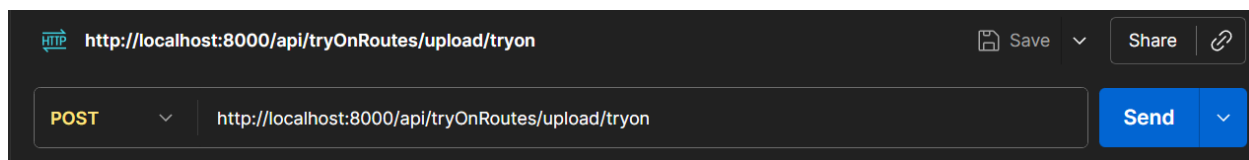
Purpose

Virtual Try-On APIs allow users to upload images and generate AI-powered try-on previews.

Upload User Image

Endpoint

POST /api/tryon/upload/tryon



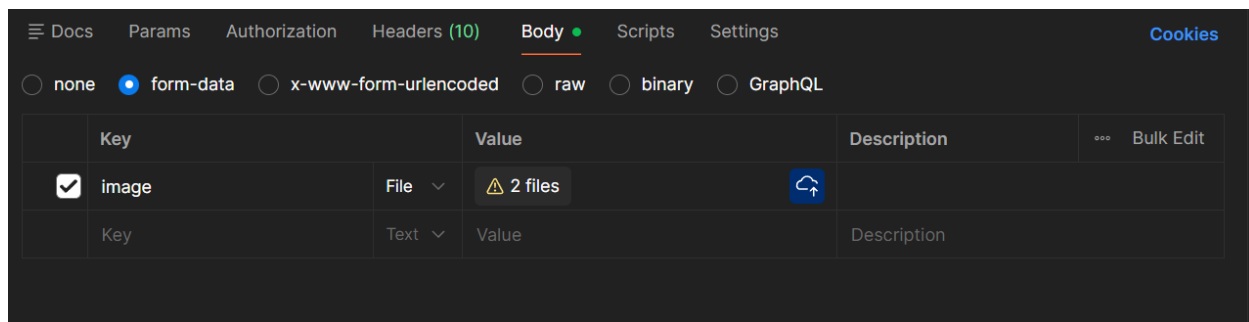
Authentication

Required

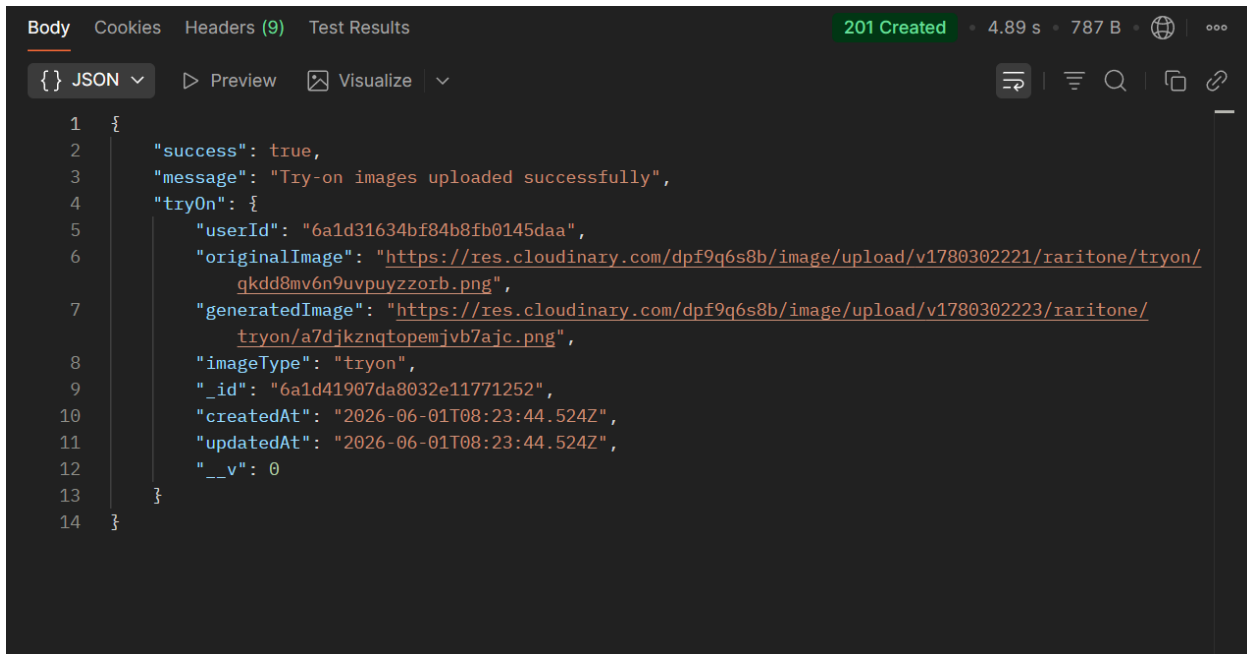
Request Body

Form Data

image: file



Success Response



The screenshot shows a REST client interface with the following details:

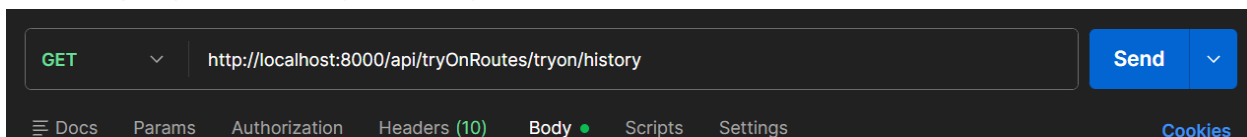
- Body** tab selected.
- JSON** format selected.
- 201 Created** status code.
- 4.89 s** execution time.
- 787 B** response size.
- Visualize** button available.

```
1  {
2    "success": true,
3    "message": "Try-on images uploaded successfully",
4    "tryOn": {
5      "userId": "6a1d31634bf84b8fb0145daa",
6      "originalImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780302221/raritone/tryon/qkdd8mv6n9uvpuyzzorb.png",
7      "generatedImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780302223/raritone/tryon/a7djknqtopemjvb7ajc.png",
8      "imageType": "tryon",
9      "_id": "6a1d41907da8032e11771252",
10     "createdAt": "2026-06-01T08:23:44.524Z",
11     "updatedAt": "2026-06-01T08:23:44.524Z",
12     "__v": 0
13   }
14 }
```

Get Virtual Try-On

Endpoint

POST / api/tryOnRoutes/tryon/history



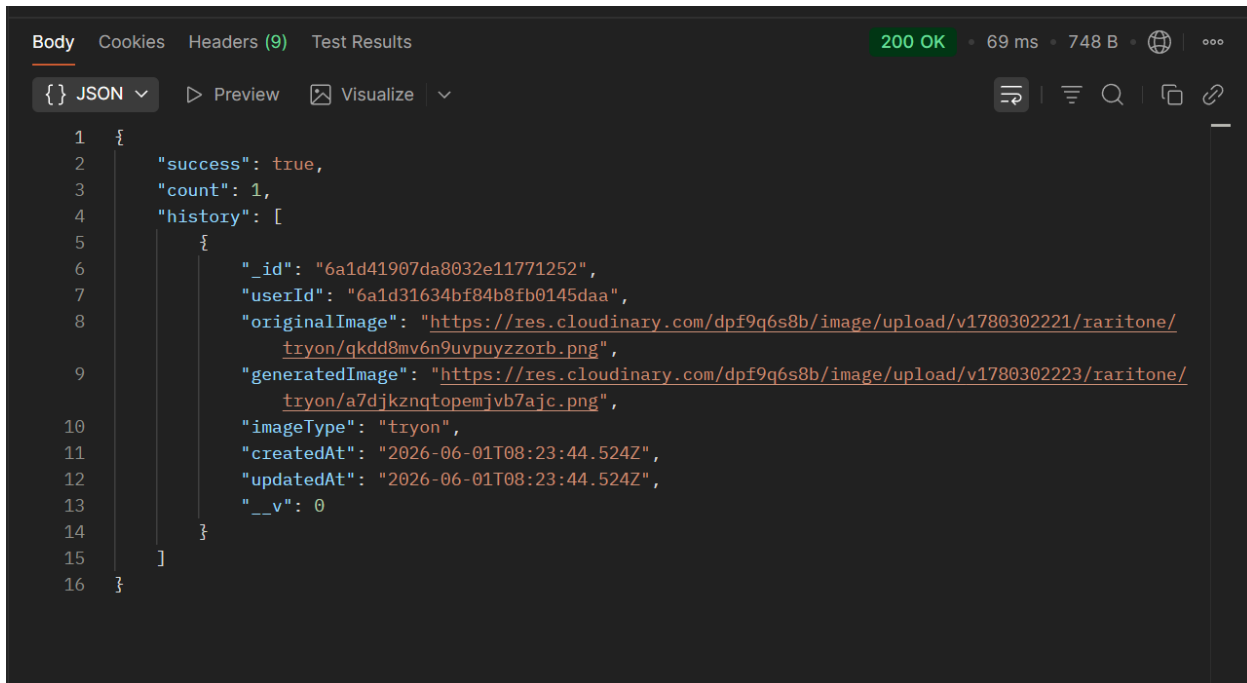
The screenshot shows a REST client interface with the following details:

- GET** method selected.
- http://localhost:8000/api/tryOnRoutes/tryon/history** endpoint.
- Send** button.
- Docs**, **Params**, **Authorization**, **Headers (10)**, **Body**, **Scripts**, **Settings**, and **Cookies** tabs are visible.

Authentication

Required

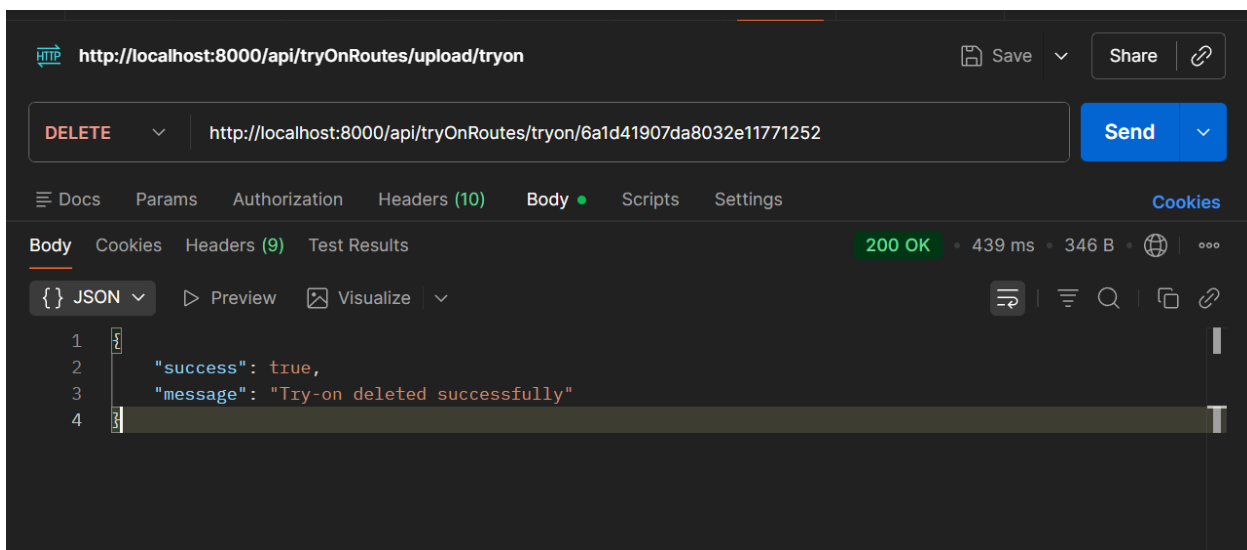
Success Response



A screenshot of a REST client interface showing a successful response. The status bar at the top indicates '200 OK' with a response time of 69 ms and a body size of 748 B. The response is displayed in JSON format, showing a success status, a count of 1, and a history array containing a single object with details about a try-on image.

```
1 {
2   "success": true,
3   "count": 1,
4   "history": [
5     {
6       "_id": "6a1d41907da8032e11771252",
7       "userId": "6a1d31634bf84b8fb0145daa",
8       "originalImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780302221/raritone/tryon/qkdd8mv6n9uvpuyzzorb.png",
9       "generatedImage": "https://res.cloudinary.com/dpf9q6s8b/image/upload/v1780302223/raritone/tryon/a7djkznqtopemjvb7ajc.png",
10      "imageType": "tryon",
11      "createdAt": "2026-06-01T08:23:44.524Z",
12      "updatedAt": "2026-06-01T08:23:44.524Z",
13      "__v": 0
14    }
15  ]
16 }
```

Delete request Virtual Try-On:



A screenshot of a REST client interface showing a successful DELETE request. The URL bar shows the endpoint 'http://localhost:8000/api/tryOnRoutes/upload/tryon'. The request method is set to 'DELETE' and the URL is 'http://localhost:8000/api/tryOnRoutes/tryon/6a1d41907da8032e11771252'. The response is displayed in JSON format, showing a success status and a message indicating the try-on was deleted successfully.

```
1 {
2   "success": true,
3   "message": "Try-on deleted successfully"
4 }
```

Validation Error Testing

The following validation scenarios were tested:

Donapati Vishnu Vardhan Reddy
Bharath kumar

Missing Required Fields

Example:

```
{  
  "height": 175  
}
```

Expected Response:

```
{  
  "success": false,  
  "message": "Required fields missing"  
}
```

Invalid Data Type

Example:

```
{  
  "height": "abc"  
}
```

Expected Response:

```
{  
  "success": false,  
  "message": "Invalid data type"  
}
```

Unauthorized Access

Expected Response:

Donapati Vishnu Vardhan Reddy
Bharath kumar

```
{  
  "success": false,  
  "message": "Unauthorized access"  
}
```

API Verification Summary

The APIs were verified for:

- Route functionality
- Request validation
- Response handling
- Database integration
- Authentication checks
- Error handling

Testing confirmed that the APIs are functioning correctly and are ready for frontend integration and future deployment.

Deliverables

Completed Deliverables:

✓ API Documentation

✓ API Verification

Donapati Vishnu Vardhan Reddy
Bharath kumar

- ✓ Postman Testing
- ✓ Success Response Testing
- ✓ Validation Error Testing
- ✓ Authentication Testing
- ✓ Postman Screenshots Collection